

Lab3

Transaction Management and Concurrency Control Techniques

Create Database Creation Script with Sample Tables and Seed Data

```
-- Create the database
CREATE DATABASE CompanyDB;
GO

-- Use the newly created database
USE CompanyDB;
GO

-- Create the Employees table
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY IDENTITY,
    FirstName NVARCHAR(50),
    LastName NVARCHAR(50),
    Department NVARCHAR(50),
    Salary DECIMAL(10, 2),
    HireDate DATE
);
GO

-- Create the Departments table
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY IDENTITY,
    DepartmentName NVARCHAR(50)
);
GO

-- Seed data for Employees table
INSERT INTO Employees (FirstName, LastName, Department, Salary, HireDate) VALUES
('John', 'Doe', 'Sales', 50000, '2020-01-15'),
('Jane', 'Smith', 'IT', 60000, '2019-02-20'),
('Michael', 'Brown', 'Sales', 55000, '2018-03-10'),
('Emily', 'Davis', 'HR', 45000, '2021-04-25'),
('Chris', 'Wilson', 'IT', 65000, '2017-05-30');
GO

-- Seed data for Departments table
INSERT INTO Departments (DepartmentName) VALUES
('Sales'),
('IT'),
('HR');
GO
```

Week 6: Transaction Management

3.4.3. Automatic Rollback

Description: Automatic rollback ensures a transaction is rolled back if an error occurs, without requiring manual intervention.

Example:

```
BEGIN TRY
    BEGIN TRANSACTION;
    -- Simulate an error
    UPDATE Employees SET Salary = Salary / 0 WHERE Department = 'Sales';
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
```

3.4.4. Nested Transactions

Description: Nested transactions allow a transaction to contain sub-transactions, which can be committed or rolled back independently.

Example:

```
BEGIN TRANSACTION;
    -- Sub-transaction 1
    BEGIN TRANSACTION;
    UPDATE Employees SET Salary = Salary * 1.1 WHERE Department = 'Sales';
    COMMIT TRANSACTION;

    -- Sub-transaction 2
    BEGIN TRANSACTION;
    UPDATE Employees SET Salary = Salary * 1.2 WHERE Department = 'IT';
    COMMIT TRANSACTION;
COMMIT TRANSACTION;
```

Week 7: Transaction Management

3.4.5. Using Save Points

Description: Save points allow a transaction to set a point to which it can roll back without affecting the entire transaction.

Example:

```
BEGIN TRANSACTION;
    -- Set save point
    SAVE TRANSACTION SavePoint1;
    UPDATE Employees SET Salary = Salary * 1.1 WHERE Department = 'Sales';

    -- Rollback to save point
    ROLLBACK TRANSACTION SavePoint1;

    -- Continue transaction
    UPDATE Employees SET Salary = Salary * 1.2 WHERE Department = 'IT';
COMMIT TRANSACTION;
```

3.4.6. Transactions in Stored Procedures

Description: Stored procedures can encapsulate transaction logic, allowing for better management and reuse of transactional code.

Example:

```
CREATE PROCEDURE UpdateSalaries
AS
BEGIN
    BEGIN TRANSACTION;
    BEGIN TRY
        UPDATE Employees SET Salary = Salary * 1.1 WHERE Department = 'Sales';
        UPDATE Employees SET Salary = Salary * 1.2 WHERE Department = 'IT';
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        SELECT ERROR_MESSAGE() AS ErrorMessage;
    END CATCH;
END;
GO

-- Call the stored procedure
EXEC UpdateSalaries;
```

Week 8: Concurrency Control Techniques

4.1. Locking

Description: Locking is a concurrency control technique that prevents multiple transactions from accessing the same data simultaneously, ensuring data consistency.

Example:

```
BEGIN TRANSACTION;
    -- Lock the Employees table
    SELECT * FROM Employees WITH (TABLOCKX);

    -- Perform operations
    UPDATE Employees SET Salary = Salary * 1.1 WHERE Department = 'Sales';
COMMIT TRANSACTION;
```

4.2. Timestamping

Description: Timestamping is a concurrency control technique that assigns timestamps to transactions to determine the order of execution and resolve conflicts.

1. **Alter Table:** We add the LastUpdatedTimestamp column to the Employees table and set its default value to the current date and time.
2. **Update Existing Records:** We initialize the LastUpdatedTimestamp for existing records.

3. **Create Trigger:** We create a trigger that automatically updates the LastUpdatedTimestamp whenever a record is updated.
4. **Transaction with Timestamp:** We declare a @Timestamp variable and use it in the UPDATE statement to ensure that only records with a LastUpdatedTimestamp less than or equal to @Timestamp are updated.

By using the LastUpdatedTimestamp, you can implement more advanced concurrency control mechanisms and ensure that your updates are based on the most recent data.

```
-- Alter the Employees table to add LastUpdatedTimestamp
ALTER TABLE Employees
ADD LastUpdatedTimestamp DATETIME DEFAULT GETDATE();
GO

-- Update the existing records with the current timestamp
UPDATE Employees
SET LastUpdatedTimestamp = GETDATE();
GO

-- Add a trigger to automatically update LastUpdatedTimestamp on updates
CREATE TRIGGER trgUpdateTimestamp
ON Employees
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE Employees
    SET LastUpdatedTimestamp = GETDATE()
    FROM Employees AS e
    INNER JOIN Inserted AS i
    ON e.EmployeeID = i.EmployeeID;
END;
GO
```

Example:

```
-- Example without explicit timestamps
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

BEGIN TRANSACTION;
    -- Perform operations
    UPDATE Employees SET Salary = Salary * 1.1 WHERE Department = 'Sales';
COMMIT TRANSACTION;

-- Example with explicit timestamps (pseudo code, as SQL Server handles this internally)
DECLARE @Timestamp DATETIME = GETDATE();

BEGIN TRANSACTION;
    -- Perform operations with consideration of @Timestamp
    UPDATE Employees SET Salary = Salary * 1.1 WHERE Department = 'Sales' AND @Timestamp
    >= LastUpdatedTimestamp;
COMMIT TRANSACTION;
```